

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Лабораторная работа №1

по дисциплине «Проектирование защищенных интеллектуальных
информационных систем»

на тему «Управление доступом с помощью списков контроля доступа.
Часть 1. Управление доступом к объектам операционных систем»

Выполнил студент гр. 321701

Ковальчук В. Н.
Самович В. М.

Проверил

Сальников Д. А.

Минск 2026

Цель: изучить механизм списков контроля доступа (ACL) и получить практические навыки настройки прав доступа к объектам файловой системы Linux.

Задание:

а) Создать группы пользователей `group_iit1`, `group_iit2`; создать пользователей `iit11`, `iit12` (группа `group_iit1`) и `iit21`, `iit22` (группа `group_iit2`); предоставить `iit21` административные привилегии; создать пользователя `iit3`.

б) Создать каталог `pzs` и вложенные каталоги `pzs11–pzs15` с правами чтения, записи и выполнения только для владельца, только для группы, только для остальных пользователей, для всех пользователей и только для администратора соответственно.

в) От имени пользователя `iit11` создать в каталогах `pzs11–pzs14` файлы, реализующие все сочетания прав (чтение, запись, выполнение и их комбинации) для каждого из четырёх классов субъектов доступа.

г) Для каждого созданного файла проверить возможность чтения, редактирования и запуска пользователями `iit11`, `iit12`, `iit21`, `iit22`, `iit3` и суперпользователем.

д) Запустить от имени `iit11` файлы, предназначенные только для выполнения, и проверить, кто из пользователей способен остановить запущенный процесс.

е) Для каждого каталога проверить возможность просмотра содержимого, создания и удаления файлов всеми пользователями.

ж) Удалить созданные файлы, каталоги, пользователей и группы.

Описание программы

Программа реализована на языке C# (.NET 9, консольное приложение) и последовательно выполняет все пункты задания: создание учётных записей и групп, создание каталога `pzs` с пятью вложенными каталогами под разные классы субъектов доступа, создание файловой матрицы прав в каждом из них, проверку прав доступа для каждой пары «пользователь – объект», запуск и попытку остановки процессов, проверку операций над каталогами и финальную очистку. Поскольку программа создаёт настоящих пользователей и групп и меняет права доступа реальной файловой системы, запускать её можно только от имени `root` – это проверяется в самом начале работы. Между шагами программа приостанавливается и ждёт нажатия клавиши, что позволяет в любой момент открыть отдельный терминал и убедиться в реальном состоянии системы (`ls -la`, `getfacl`, `id` и т. п.), прежде чем перейти к следующему шагу.

Классические биты `chmod` достаточны, чтобы выдать права владельцу, «остальным» или всем сразу, однако они не позволяют выдать права ровно одной именованной группе, не затронув при этом остальных, – группа файла в POSIX всего одна. Поэтому для класса «группа» классические биты обнуляются (`chmod 000`), а затем командой `setfacl` добавляется именованная ACE для `group_iit1`: это и есть собственно список контроля доступа, а не просто традиционные три класса прав. Соответствующий фрагмент кода показан ниже (см. листинг 1).

```
1 public static void ApplyRule(string path, int subject, int permIndex)
2 {
3     var octal = Config.PermOctals[permIndex];
4     var sym = Config.PermSyms[permIndex];
5
6     Report.Execute($"Reset ACL on '{path}'", "setfacl", "-b", path);
7
8     switch (subject)
9     {
10         case 1:
11             Report.Execute($"Apply access rule (owner, {sym}) on '{path}'", "chmod",
12                 ↪ $"{octal}00", path);
13             break;
14         case 2:
15             Report.Execute($"Clear classic bits on '{path}'", "chmod", "000", path);
16             Report.Execute($"Apply access rule (group 'group_iit1', {sym}) on '{path}'",
17                 ↪ "setfacl", "-m", $"g:group_iit1:{sym}", path);
18             break;
19         case 3:
20             Report.Execute($"Apply access rule (other, {sym}) on '{path}'", "chmod",
21                 ↪ $"00{octal}", path);
22             break;
23         case 4:
24             Report.Execute($"Apply access rule (all, {sym}) on '{path}'", "chmod",
25                 ↪ $"{octal}{octal}{octal}", path);
26             break;
27         case 5:
28             Report.Execute($"Set owner of '{path}' to root", "chown", "root:root", path);
29             Report.Execute($"Apply access rule (administrator only, {sym}) on '{path}'",
30                 ↪ "chmod", $"{octal}00", path);
31             break;
32     }
33 }
```

Листинг 1 – Применение правила доступа к объекту файловой системы

Результаты выполнения

Ниже приведён фрагмент реального вывода программы, запущенной в изолированном Linux-контейнере (см. листинг 2). Он показывает, что каталог `pzs12` доступен на чтение/запись только членам группы `group_iit1` (`iit11`, `iit12`), тогда как `iit21`, `iit22`, `iit3` получают отказ, а `root` проходит любую проверку.

Отдельного внимания заслуживает файл `pzs14/file21`: он принадлежит `iit11`, а право на чтение по ACL выдано группе `group_iit1`, в которую `iit11` тоже входит. Тем не менее `iit11` получает отказ, а `iit12` – доступ. Это в точности соответствует правилу выбора ACE, описанному в теоретической части задания: если запрашивающий – владелец объекта, то используется только запись владельца (обнулённая `chmod 000`), а именованная запись группы для него не рассматривается вовсе, даже если он формально состоит в этой группе.

Также подтвердилось, что ни один из файлов вида `filex5` (право только на выполнение, без права на чтение) не удаётся запустить ни одному пользователю, включая владельца `iit11`: интерпретируемый `sh/bash`-скрипт запускается не самим ядром, а внешним интерпретатором, которому требуется прочесть содержимое файла, – права на одно только выполнение для этого недостаточно (в отличие от скомпилированных бинарных файлов).

```

1 == Step 1: Create groups group_iit1, group_iit2 ==
2 [OK] Create group 'group_iit1' $ groupadd group_iit1
3 [OK] Create group 'group_iit2' $ groupadd group_iit2
4
5 == Step 2: Create users iit11, iit12 in group_iit1 ==
6 [OK] Create user 'iit11' $ useradd -m -G group_iit1 iit11
7 [OK] Create user 'iit12' $ useradd -m -G group_iit1 iit12
8 ...
9 == Step 4: Grant iit21 administrative privileges ==
10 [OK] Grant admin privileges to 'iit21' (group 'sudo') $ usermod -aG sudo iit21
11
12 == Step 7: Create folders pzs11..pzs15 (owner/group/other/all/admin, rwx) ==
13 [OK] Create directory '/acl_lab/pzs/pzs11' $ mkdir -p /acl_lab/pzs/pzs11
14 [OK] Apply access rule (owner, rwx) on '/acl_lab/pzs/pzs11' $ chmod 700
    ↪ /acl_lab/pzs/pzs11
15 [OK] Create directory '/acl_lab/pzs/pzs12' $ mkdir -p /acl_lab/pzs/pzs12
16 [OK] Clear classic bits on '/acl_lab/pzs/pzs12' $ chmod 000 /acl_lab/pzs/pzs12
17 [OK] Apply access rule (group 'group_iit1', rwx) on '/acl_lab/pzs/pzs12' $ setfacl -m
    ↪ g:group_iit1:rwx /acl_lab/pzs/pzs12
18 ...
19
20 -- Проверка каталога pzs12 (доступ только для группы group_iit1 через ACL) --
21 [ALLOWED] iit11 ListDirectory /acl_lab/pzs/pzs12
22 [ALLOWED] iit12 ListDirectory /acl_lab/pzs/pzs12
23 [DENIED ] iit21 ListDirectory /acl_lab/pzs/pzs12
24 [DENIED ] iit22 ListDirectory /acl_lab/pzs/pzs12
25 [DENIED ] iit3 ListDirectory /acl_lab/pzs/pzs12
26 [ALLOWED] root ListDirectory /acl_lab/pzs/pzs12
27
28 -- Файл pzs14/file21: владелец iit11, ACE группы group_iit1 = r-- --
29 [OK] Apply access rule (group 'group_iit1', r--) on '/acl_lab/pzs/pzs14/file21' $
    ↪ setfacl -m g:group_iit1:r-- /acl_lab/pzs/pzs14/file21
30 [DENIED ] iit11 Read /acl_lab/pzs/pzs14/file21
31 [ALLOWED] iit12 Read /acl_lab/pzs/pzs14/file21
32 [DENIED ] iit21 Read /acl_lab/pzs/pzs14/file21
33
34 == Step 15: Run every filex5 script as iit11, then try to stop it as every identity ==
35 [FAIL] Start '/acl_lab/pzs/pzs11/file15' as 'iit11' -sh: 1: /acl_lab/pzs/pzs11/file15:
    ↪ Permission denied
36 [FAIL] Start '/acl_lab/pzs/pzs14/file45' as 'iit11' -sh: 1: /acl_lab/pzs/pzs14/file45:
    ↪ Permission denied
37 ...
38
39 === Summary ===
40 Operations: 594 total, 20 failed
41 Read allowed 181 / 600 denied 419
42 Write allowed 181 / 600 denied 419
43 Execute allowed 94 / 600 denied 506
44 ListDirectory allowed 23 / 36 denied 13
45 CreateEntry allowed 18 / 36 denied 18
46 DeleteEntry allowed 18 / 36 denied 18

```

Листинг 2 – Фрагмент вывода программы

Выводы

В ходе выполнения лабораторной работы был освоен механизм списков контроля доступа: разработанная программа настраивает права доступа к каталогам и файлам для всех предусмотренных заданием классов субъектов (владелец, группа, остальные, все, администратор) и экспериментально подтверждает их корректность прямыми проверками от имени каждого пользователя. Практически показано, что классических прав `rwX` достаточно не всегда – выдать права ровно одной именованной группе можно только через ACL, добавляемый утилитой `setfacl`.

Полученные результаты подтвердили и семантическую составляющую теоретической части задания: доступ определяется единственной, наиболее подходящей ACE, а для владельца объекта именованная запись группы не учитывается вовсе, даже если он входит в эту группу. С прагматической точки зрения работа показала практическую границу применимости разрешения на выполнение – оно защищает только скомпилированные программы, а для исполняемых скриптов бесполезно без права на чтение, поскольку их читает интерпретатор, а не ядро.